기존 IDS의 한계 · 새 공격과 제한된 라벨

침입 탐지 시스템 (IDS) · 통신 기록의 특징으로 정상·공격 유형을 구분하는 시스템

운영 시작 시 확보한 공격 라벨만으로 이후 등장하는 공격까지 대응하기 어려운 문제

입력의 변화

네트워크·서비스·공격의 다양성
학습 시점과 다른 트래픽의 등장

제한된 공격 라벨

새 공격 조사 후 확보하는 사례
충분한 학습 자료 확보의 어려움

오탐과 갱신 부담

정상 트래픽에 대한 잘못된 경보
새 사례 반영에 필요한 추가 계산

고정된 클래스의 한 번의 점수에 더해, 새 라벨을 반영한 뒤의 탐지 품질과 비용 확인

Sommer · Paxson, Outside the Closed World: On Using Machine Learning for Network Intrusion Detection, IEEE S&P 2010 · 학습 자료 부족·입력 변동성·오류 비용

공격 수집 기록을 단계별 라벨 공급으로 활용

CSE-CIC-IDS2018 · AWS 시험망에서 정상 동작과 계획된 공격을 실행·수집

CSE · UNB CIC, CSE-CIC-IDS2018 on AWS, 2018 · 공식 문서의 공격 일정 Table 2

2/14	2/15·16	2/20·21	2/22·23	2/28·3/1	3/2
Brute force	DoS	DDoS	Web attacks	Infiltration	Bot

수집된 공격 유형을 라벨 확보 단계로 구성 · 같은 train에서 단계별 사례 선택

초기 사례 1,000개

정상 750개
Brute force·DoS 각 125개

신규 공격당 n개

DDoS · Web · Infiltration · Bot
각 단계의 신규 유형에서 n행 선택

단계별 탐지기 갱신

같은 사례로 XGB/PFN 갱신
고정 test에서 품질·비용 비교

ToN-IoT도 자체 train에서 같은 절차로 독립 평가 · UNSW Canberra, The TON_IoT Datasets

TabPFN을 검토하는 이유 · 가중치 재학습 없는 적응

새 공격 반영 시 가중치 재학습을 생략할 수 있는 in-context learning 활용

Grinsztajn et al., TabPFN-3: Technical Report, arXiv 2026 · 합성 표 과제로 사전학습

Context · 라벨 사례

기존 정상·공격 + 신규 공격
특징과 정답을 함께 입력

Query · 분류할 통신 기록

특징만 입력 · 정답은 평가에 사용

→

TabPFN v3

사전학습 가중치 고정

→

정상·공격별 확률

context를 참조한 추론

채택 이유	사전학습된 예측 규칙을 활용 · 라벨 사례 입력으로 현재 IDS 문제에 적응
검증할 이점	소량 라벨·제한된 보유량에서 품질을 유지하며 갱신 비용을 줄일 가능성

가중치 fine-tuning은 별도 최적화 절차 · 이번 실험은 미수행 / context 전처리·캐시 준비 시간은 측정

평가 목적 · 같은 사례로 비교하는 모델과 보유 방식

새 공격의 라벨 사례가 소량 확보될 때
누적 재학습과 context 갱신의 탐지 품질·계산 비용 비교

공통 입력 · 초기 1,000개 + 단계별 신규 공격 라벨 / 모든 방법에 동일 사례 제공

실제 사용하는 사례	XGB	TabPFN
누적 · 지금까지 제공한 전부	누적 사례로 재학습	누적 사례로 context 갱신
1,000개 · FIFO / 무작위 / 유사도	같은 버퍼 행으로 재학습	같은 버퍼 행으로 context 갱신

탐지 품질

전체·신규 공격별 F1

기존 클래스 유지

동일 평가 사례의 갱신 전후 변화

갱신 비용

초기 준비·추론과 분리한 시간

평가 범위 · 신규 공격의 라벨 확보 후 분류 / 라벨 없는 미지 공격 탐지와 네트워크 간 전이는 별도 과제

평가 방법 · 데이터 출처와 사례 선택

각 벤치마크 안에서 train의 행을 선택 · CIC와 ToN 사이의 행 결합 없음

CIC2018

수집 벤치마크

정제 후 train / test 분리

상충 라벨 제거
학습·평가 행 분리

Train → 초기 1,000개 + 신규 공격당 n개 선택

Test → 고정 3,042,473개 · 학습/context에 미사용

ToN-IoT

수집 벤치마크

정제 후 train / test 분리

상충 라벨 제거
학습·평가 행 분리

Train → 초기 1,000개 + 신규 공격당 n개 선택

Test → 고정 2,271,723개 · 학습/context에 미사용

제공 사례 · 같은 train에서 단계별로 선택한 라벨 사례 / 별도 출처의 데이터셋 생성 없음

실제 사용 사례 · 누적은 지금까지 제공한 전부 / 버퍼는 삭제 후 남긴 사례와 신규 사례 1,000개

평가 방법 · 단계별 공급량과 실제 보유량

n = 5 · 10 · 20 · 30 · 50 · 100개 / 신규 공격 클래스 · seed 43·44·45

n은 확보 가능한 라벨 수의 실험 조건 · 하루 공격량이나 최적 표본 수를 뜻하지 않는 값

CIC2018

단계	제공 클래스	제공 수	누적 보유	버퍼
1	Benign Brute force DoS	750 125 125	1,000	1,000
2	DDoS	n	1,000+n	1,000
3	Web attacks	n	1,000+2n	1,000
4	Infiltration	n	1,000+3n	1,000
5	Bot	n	1,000+4n	1,000

ToN-IoT

단계	제공 클래스	제공 수	누적 보유	버퍼
1	Benign DoS Scanning	750 125 125	1,000	1,000
2	DDoS Injection	n n	1,000+2n	1,000
3	Password	n	1,000+3n	1,000
4	XSS	n	1,000+4n	1,000
5	Backdoor Ransomware	n n	1,000+6n	1,000
6	MITM	n	1,000+7n	1,000

제공 수는 클래스당 사례 수 · 누적·버퍼 모두 XGB/PFN 공통 · 표의 보유량은 매 단계 실제 사용 행 수

평가 흐름 · 사례 보유에서 모델 갱신과 test까지

예시 · CIC 3단계, 신규 공격당 100개 · 같은 사례를 서로 다른 방식으로 갱신

라벨 확보 순서

초기 정상 750개 · 공격 250개

DDoS Web



같은 고정 test

보유 방식 선택 · XGB/PFN 공통

누적



1,200개 · 기존 사례 모두 보유

버퍼



1,000개 · 기존 삭제 + 신규 전량

버퍼 예시: 무작위, seed 43 · 막대 길이 = 행 수

XGB · 매 단계 모델 재학습

트리 구조·리프 출력 학습

해당 단계의 보유 사례로 새 모델 구성

예측·평가

PFN · 사전학습 가중치 유지

context
캐시 갱신

TabPFN
가중치 고정

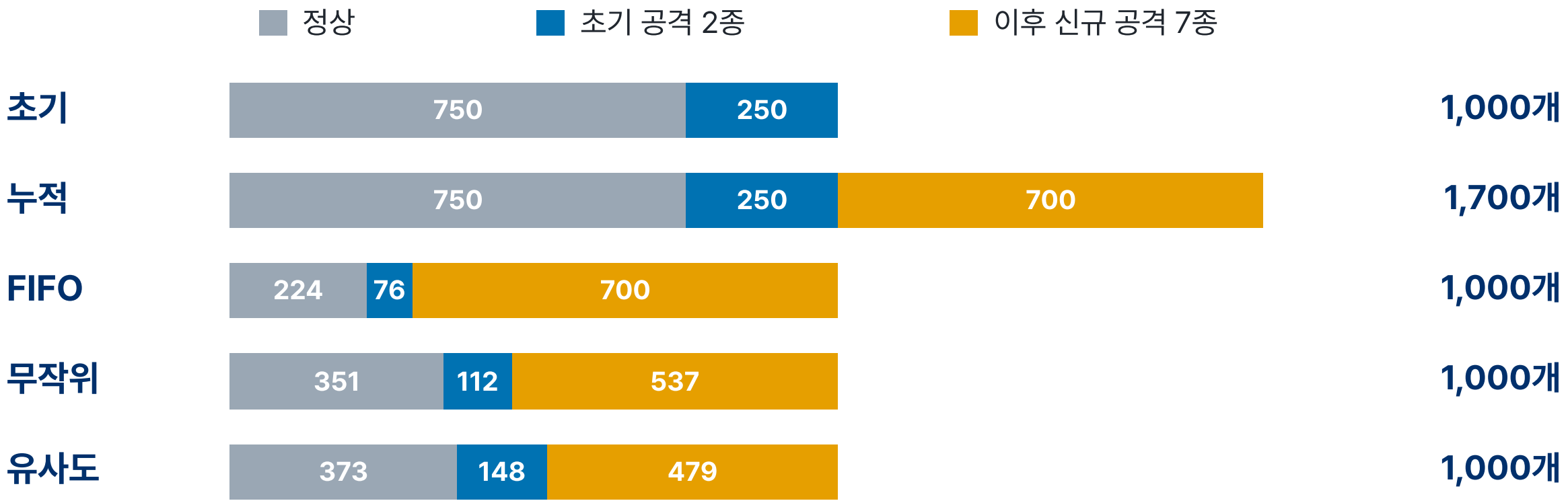
예측·평가

점선: 매 단계 갱신 영역 · 전체 test는 마지막 모델에만 평가

각 단계: 갱신 시간·고정 패널 확인 / 마지막 단계: 전체 test 품질 평가

버퍼 교체에 따른 클래스 구성 변화

예시 · ToN 최종 단계, 신규 공격당 100개, seed 43 · 각 막대는 실제 보유 행 수



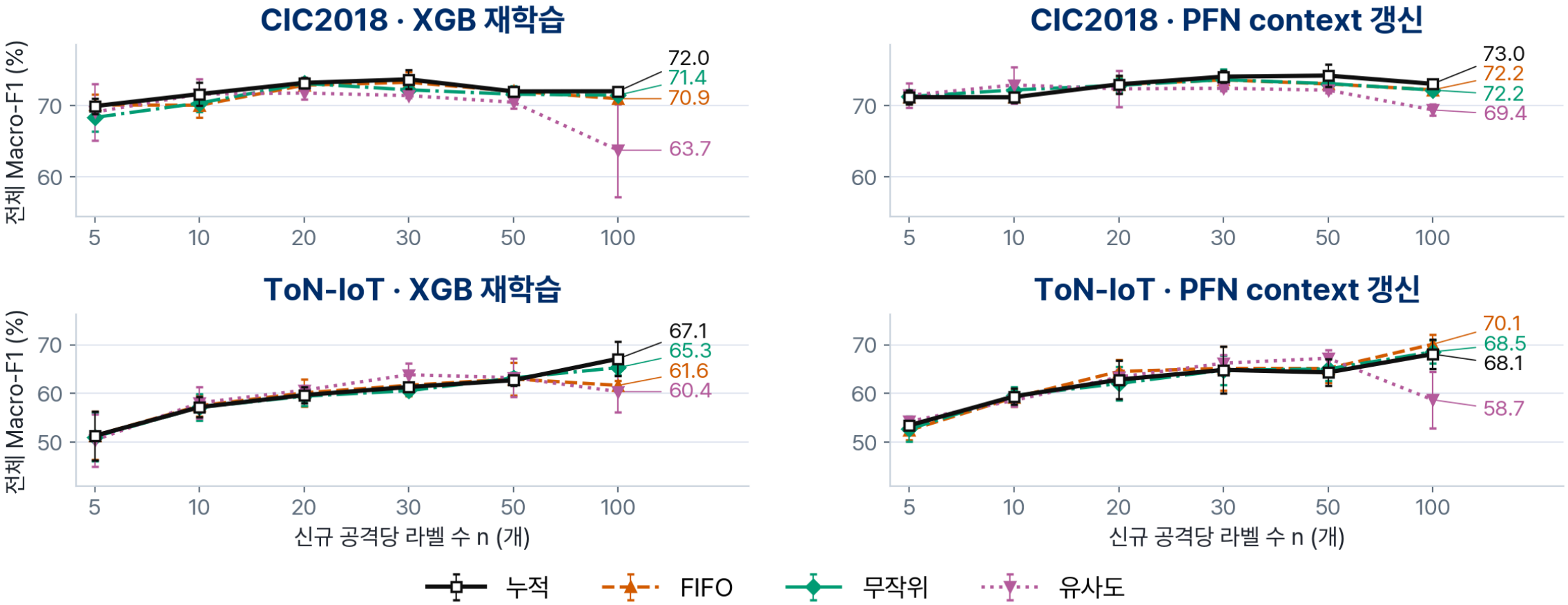
정상 비율 · 초기 75.0% → 누적 44.1% / FIFO 22.4% / 무작위 35.1% / 유사도 37.3%

삭제 기준 · FIFO: 먼저 확보한 행 / 무작위: 임의의 행 / 유사도: 신규와 가까운 행 · 클래스별 최소 1개 보존

결과 · 전체 클래스의 최종 분류 성능

최종 모델 · 전체 고정 test · 모든 클래스 F1의 산술평균

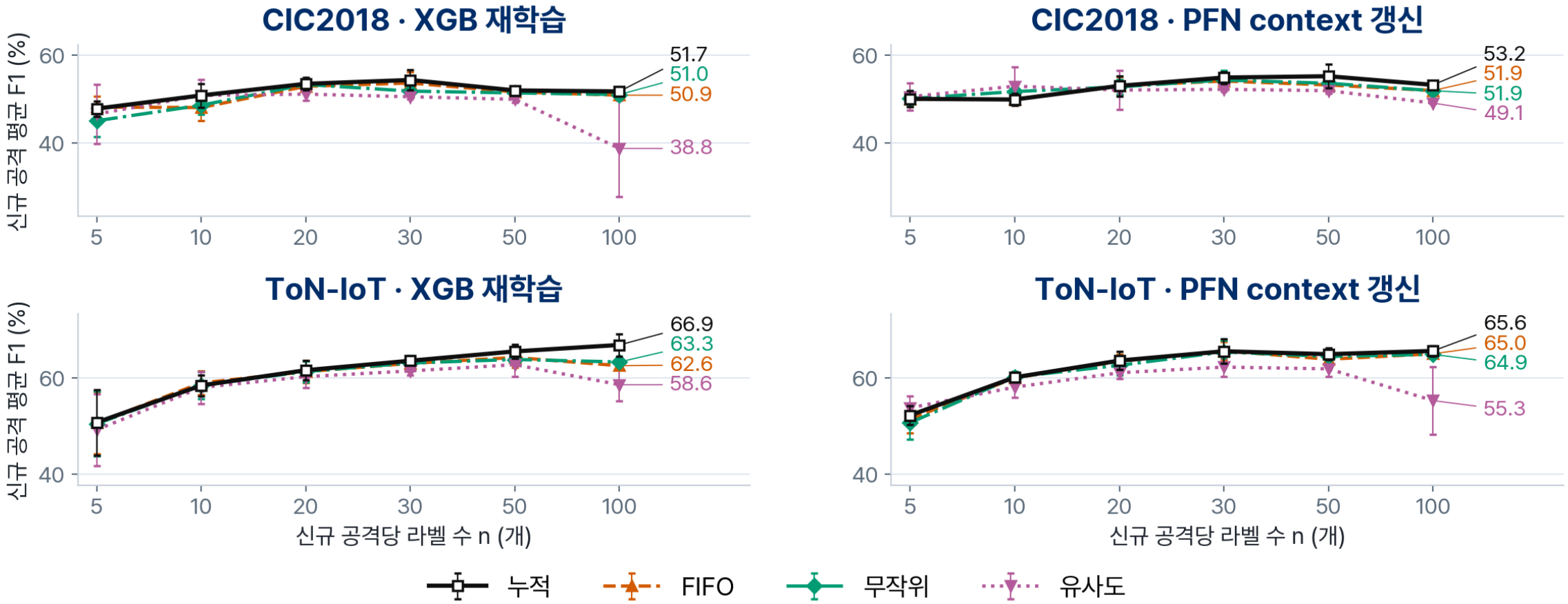
평균 ± 표준편차 · seed 43·44·45 · 같은 데이터셋의 두 패널은 같은 축 · 우측 숫자: n=100



결과 · 신규 공격 라벨 수에 따른 탐지 품질

최종 모델 · 전체 test에서 계산한 신규 공격 클래스별 F1의 산술평균

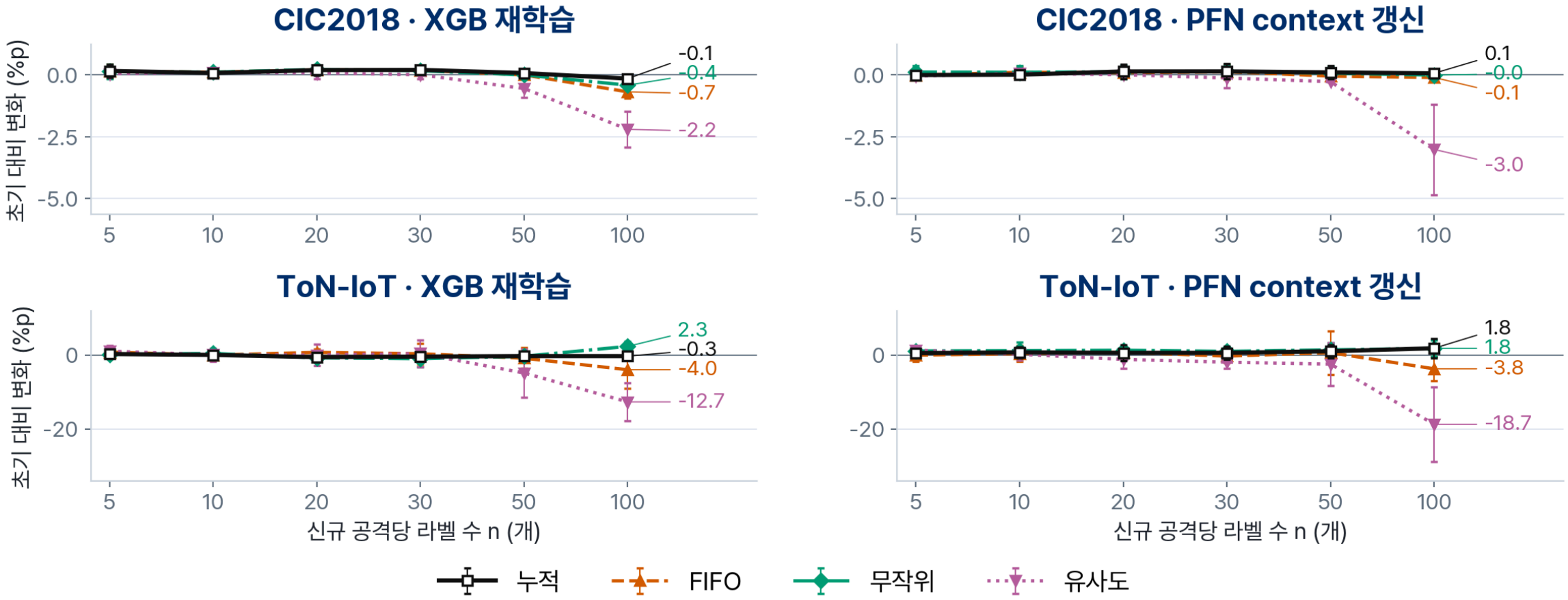
CIC 4개 · ToN 7개 신규 공격 · 정상·초기 공격에서 발생한 오탐도 F1에 포함 · 3 seed 평균 ± 표준편차



결과 · 갱신 이후 초기 클래스 성능 변화

초기 정상·공격 2종의 같은 평가 표본 · 최종 − 초기 Macro-F1 (%p)

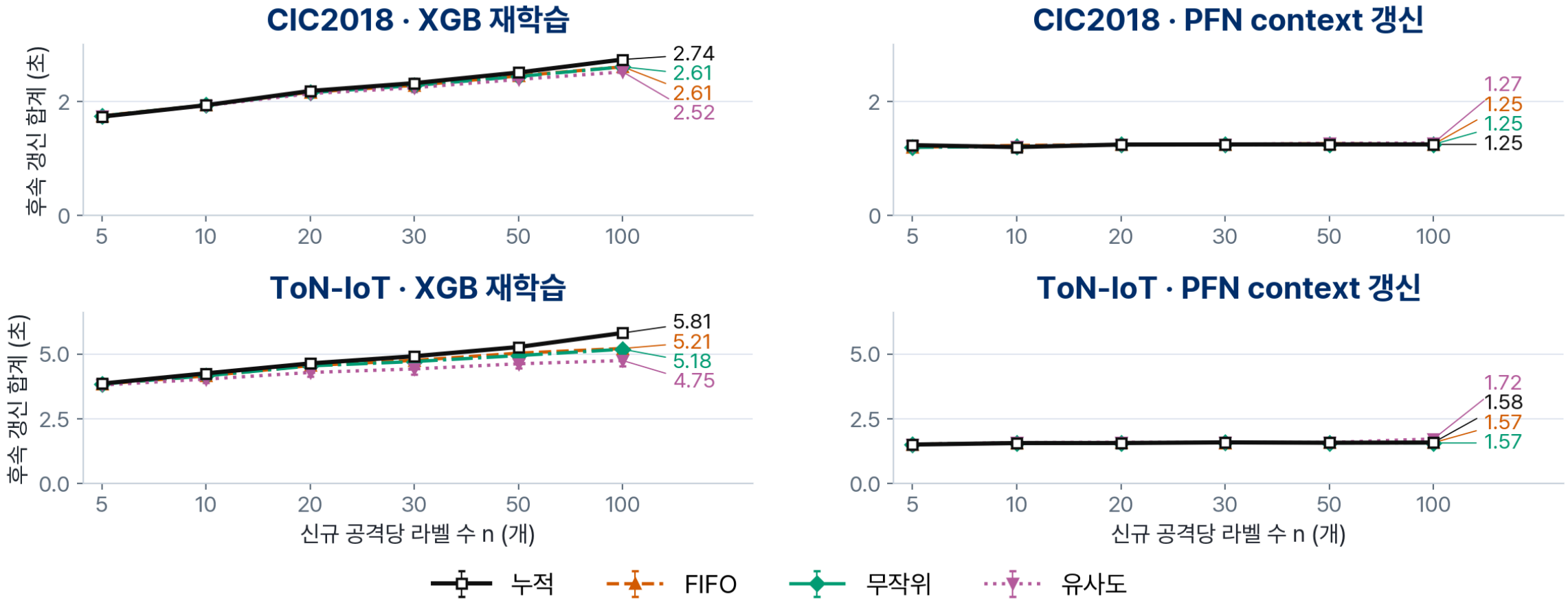
클래스당 최대 2,000개 고정 test 표본 · 0 아래는 초기보다 성능 하락 · 3 seed 평균 ± 표준편차



결과 · 같은 사례를 반영하는 후속 갱신 비용

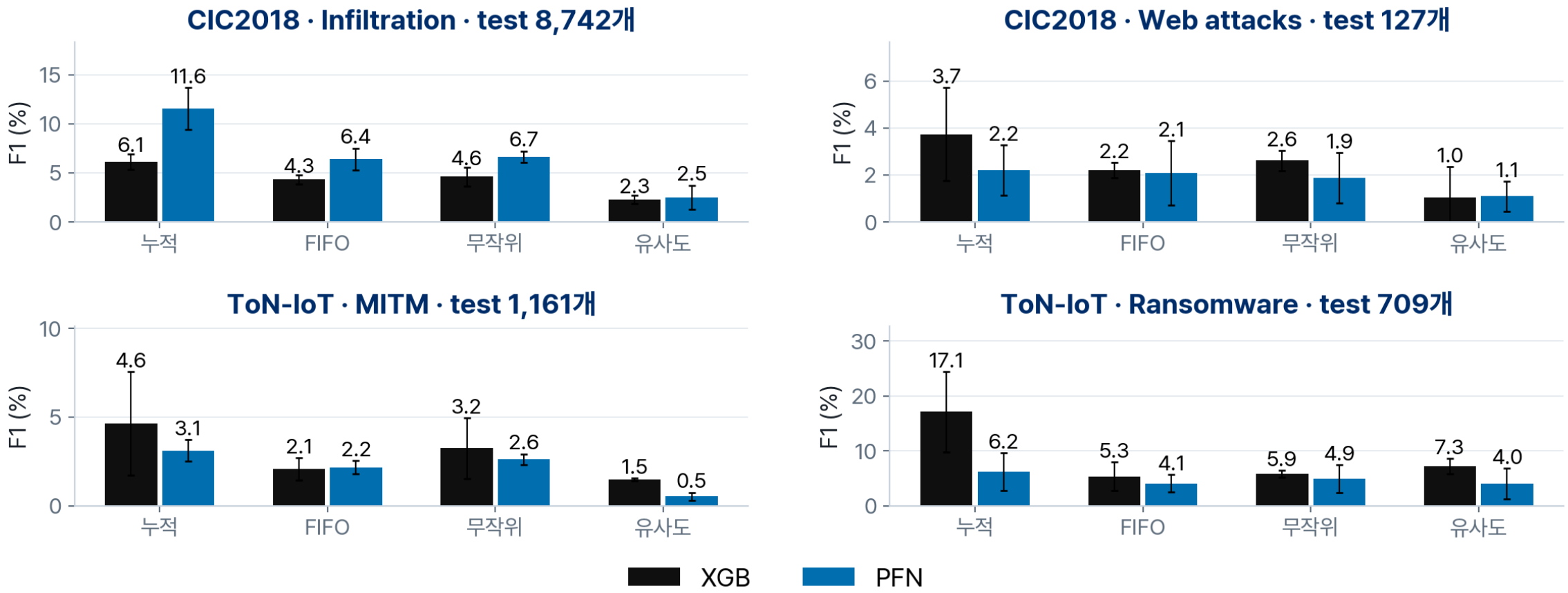
2단계 이후 갱신 시간 합계 · 초기 준비와 모든 추론 제외

사례 선택·삭제 + XGB 재학습 또는 PFN 전처리·캐시 준비 · 같은 장비, 별도 실행 · 3 seed 평균 ± 표준편차



결과 해석 · 같은 버퍼에서도 남는 희소 공격 오류

n = 100 · test에서 가장 희소한 공격 2종씩 · 3 seed 평균 ± 표준편차



ToN Ransomware · PFN 무작위 평균 재현율 96.7% / 정밀도 2.5% · 다음 분석: 오탐의 출처와 보정 효과

부록 · CIC2018 클래스별 성능

n = 100 · 마지막 모델 전체 test · F1 (%)의 3 seed 평균 · 평가 표본 수 내림차순

		XGB 재학습				PFN context 갱신			
클래스	test 수	누적	FIFO	무작위	유사도	누적	FIFO	무작위	유사도
Benign	2,652,203	98.14	96.74	97.18	91.96	99.35	98.98	99.08	92.01
DDoS	262,680	99.44	99.42	99.23	79.90	99.90	99.89	99.89	97.68
Bot	41,542	97.63	97.58	97.68	72.06	99.24	99.34	99.19	95.15
DoS	39,483	98.92	98.84	98.89	98.97	98.96	98.60	98.40	97.23
Brute force	37,696	99.93	97.53	99.93	99.94	99.97	99.97	99.97	99.97
Infiltration	8,742	6.14	4.32	4.61	2.29	11.56	6.41	6.66	2.50
Web attacks	127	3.74	2.20	2.61	1.03	2.21	2.09	1.88	1.10

같은 정책·같은 seed에서 XGB/PFN의 행 ID 일치 · 전체 n의 수치와 표준편차는 실험 보고서에 수록

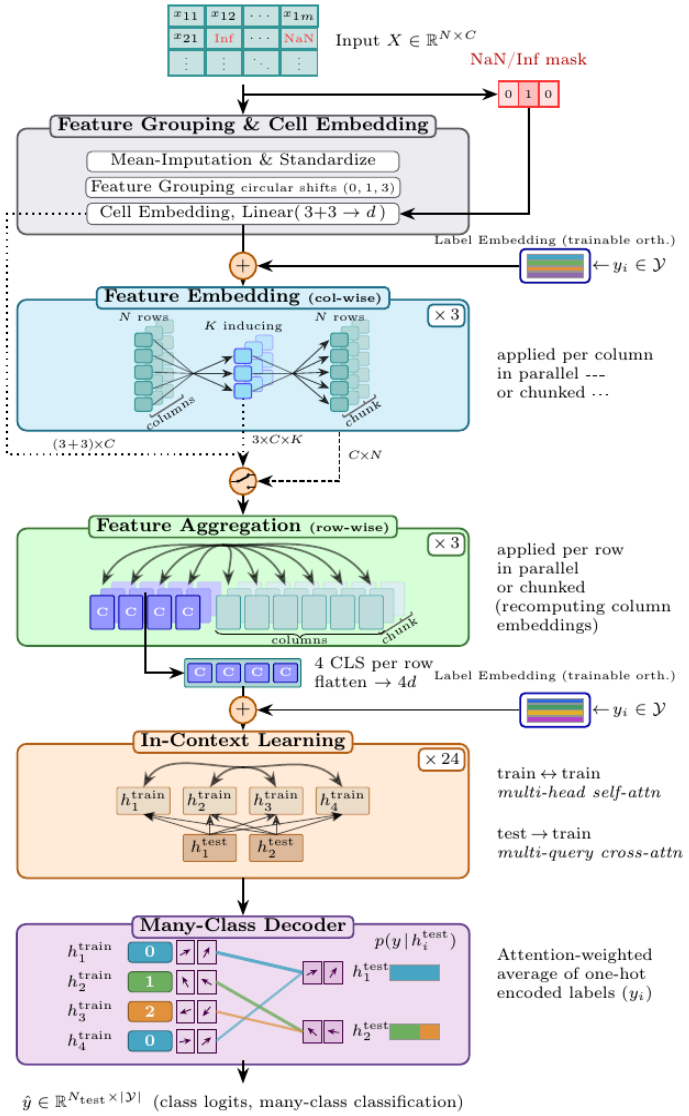
부록 · ToN-IoT 클래스별 성능

n = 100 · 마지막 모델 전체 test · F1 (%)의 3 seed 평균 · 평가 표본 수 내림차순

		XGB 재학습				PFN context 갱신			
클래스	test 수	누적	FIFO	무작위	유사도	누적	FIFO	무작위	유사도
Benign	1,474,302	91.98	85.87	90.77	87.48	90.69	89.24	90.81	70.44
XSS	343,854	83.77	82.88	83.12	77.69	83.44	84.89	84.28	76.60
Password	210,951	89.59	87.30	89.37	80.74	89.83	90.05	90.04	80.09
DDoS	103,338	92.01	91.74	90.86	76.92	94.34	94.69	93.18	62.56
Injection	68,891	81.94	69.52	74.17	69.11	83.08	80.08	79.62	65.20
Backdoor	40,030	99.16	99.08	96.80	97.04	99.46	99.38	99.80	98.10
DoS	22,458	73.80	78.28	82.82	89.20	78.96	91.42	84.05	89.95
Scanning	6,029	37.24	14.25	35.91	16.63	51.71	65.05	56.11	39.12
MITM	1,161	4.63	2.08	3.25	1.48	3.12	2.17	2.62	0.54
Ransomware	709	17.11	5.34	5.86	7.25	6.19	4.06	4.94	4.02

같은 정책·같은 seed에서 XGB/PFN의 행 ID 일치 · 전체 n의 수치와 표준편차는 실험 보고서에 수록

부록 · TabPFN v3의 기록 표현



1 입력값 표현

통신 기록의 각 값을 벡터로 변환
결측 표시와 context 라벨 정보 반영

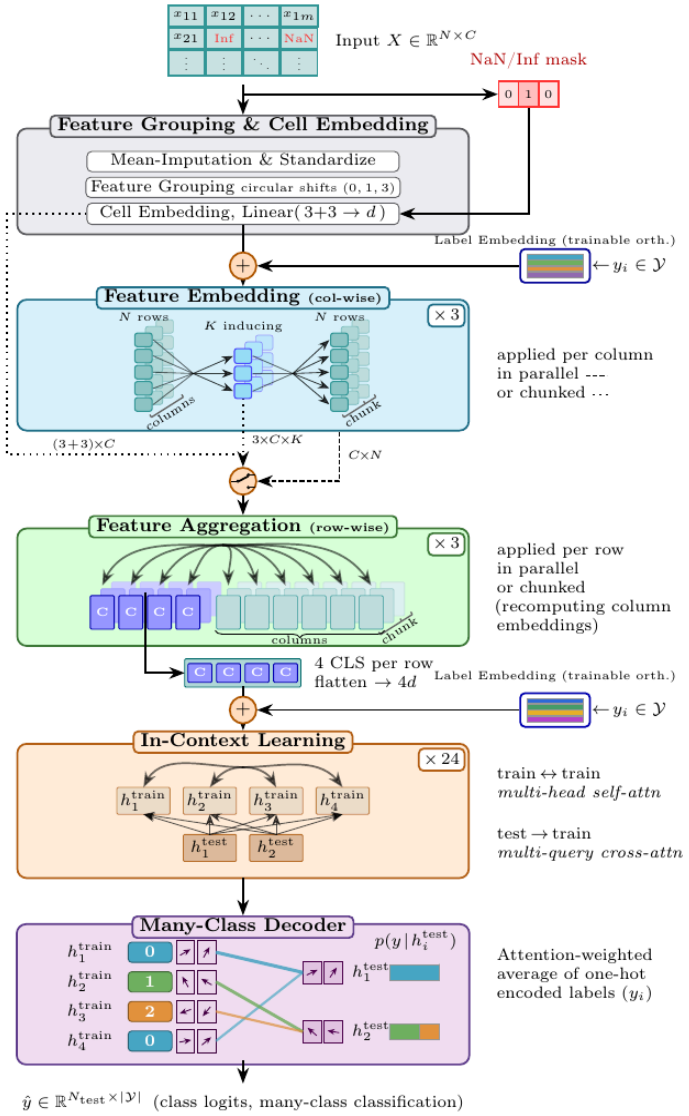
2 열의 분포 요약

여러 기록에서 같은 특징을 살펴
그 특징의 분포를 표현

3 한 기록의 특징 통합

여러 열을 행 단위 표현으로 압축
다음 단계에서 사례끼리 비교

부록 · TabPFN v3의 추론과 fine-tuning



4 Context를 참조한 예측

query가 라벨이 있는 기록을 참조
attention으로 참고 비중 계산

5 라벨을 이용한 확률 출력

context 라벨을 가중합하는 decoder
기존·신규 클래스 사례 모두 필요

별도의 가중치 fine-tuning

라벨 사례를 context와 query로 나눠 query 오차 역전파
이번 실험은 미수행 · context/cache 갱신만 사용